

CS 486/686

Pretraining Language Models From First Principles

Yuntian Deng

Lecture 20

How a transformer learns to predict text

Search ›

Uncertainty ›

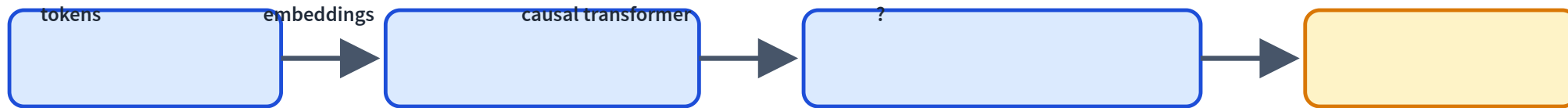
Decisions ›

Learning

Learning goals

- Turn **text** into tokens and embeddings.
- Explain **next-token prediction** and **cross-entropy**.
- Explain **pretraining** as self-supervised learning at scale.
- Distinguish **training** from **decoding / inference**.

From architecture to objective



Architecture alone does nothing. We need a training signal.

The language modeling task

The robot picked up the cup because it was ____

empty

likely

blue

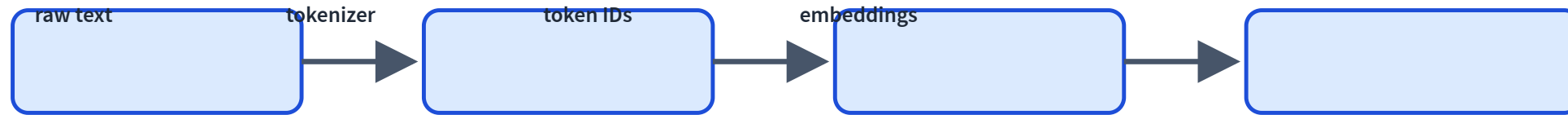
possible

quantum

unlikely

Language modeling means predicting what comes next.

Text is not directly a tensor



Models see token IDs and vectors, not words as humans see them.

Why tokenization is hard

cat

cats

unbelievable

CS486

spaces?

word vocab

brittle for rare words

character vocab

very long sequences

Subwords are the compromise

Common pieces become reusable tokens.

unbelievable

un

believe

able

Exact splits depend on the tokenizer, but the idea is the same: balance vocabulary size and sequence length.

Tokenizer playground LIVE

See how real text splits into subword tokens and IDs.

A tokenizer splits text into **subword** pieces, each with an integer id. The dot (·) marks a leading space.

"unbelievable"

"The robot picked up the cup."

"CS486 tokenization isn't trivial"

"transformers"

unbelievable

tokenize

un

403

bel

6667

iev

11203

able

540

12 characters

4 tokens

Common words stay whole; rare words split into reusable pieces (un·bel·iev·able).

Token IDs become embeddings

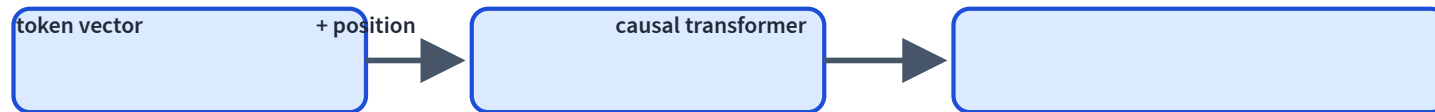
token	id	embedding row
The	791	[0.12, -0.40, ...]
robot	21933	[0.51, 0.08, ...]
cup	7721	[-0.20, 0.33, ...]

This is the embedding table from L18.

Pretraining updates these rows.

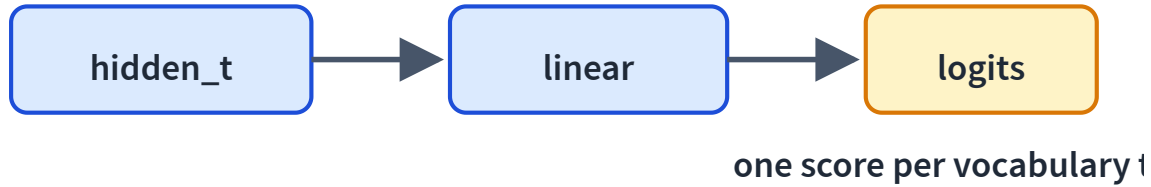
Add position, then transformer blocks

$$\text{input}_t = \text{token_embedding}_t + \text{position}_t$$



Each position can only use earlier context because of the causal mask.

The output head predicts a token



At every position, the model outputs a vector of scores.

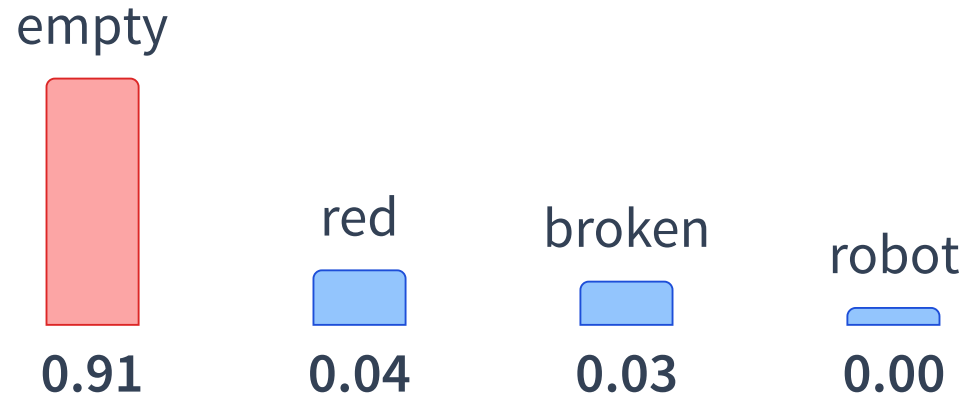
If the vocabulary has 150k tokens, there are 150k logits.

Logits are scores, not probabilities

candidate next token	logit
empty	8.2
red	3.1
broken	2.7
robot	-1.4

Bigger logit = more preferred, but not yet a probability.

Softmax turns logits into probabilities



Now we have a next-token distribution.

One sequence gives many examples

Raw text supervises itself by shifting one token right.

input context	target
The	robot
The robot	picked
The robot picked	up
The robot picked up the	cup

Causal masking makes training legal

query

key position			
	x	x	x
		x	x
			x

Position t predicts token $t + 1$.

It may only see tokens $\leq t$.

No future peeking, even during training.

Autoregressive factorization

A text probability is built one next-token prediction at a time:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Same idea as generation: left to right, token by token.

Cross-entropy for the next token

If the true next token is **empty**:

$$\text{loss} = -\log P(\text{empty} \mid \text{context})$$

This is multiclass classification over the vocabulary.

The loss, token by token

LIVE MODEL

Each bar is $-\log p(\text{actual next token})$ — one sentence, many signals.

The robot picked up the cup becau:

score my own text (real model)

- robot  15.3
 - picked  8.2
 - up  0.4
 - the  1.5
 - cup  7.9
 - because 6.8
 - it  1.1
 - was  1.2
 - empty  6.1
- average loss = 5.37 nats/token
low = predictable ("up" after "picked"); high = surprising

The pretraining loop

```
ids = tokenizer(text)
x = ids[:-1]
y = ids[1:]

logits = model(x)                # [T, vocab_size]
loss = cross_entropy(logits, y)
loss.backward()
optimizer.step()
```

This is L17's training loop, scaled to billions/trillions of tokens.

What gets updated?

embeddings

token vectors

attention

retrieval
weights

feed-forward

per-token
compute

output head

logits

Pretraining changes all model parameters.

Why this is self-supervised

No human labels are required.

Raw text gives both inputs and targets by shifting tokens.

Scale becomes possible because labels are essentially free.

But data quality still matters.

What does pretraining teach?

patterns

grammar, style, syntax

associations

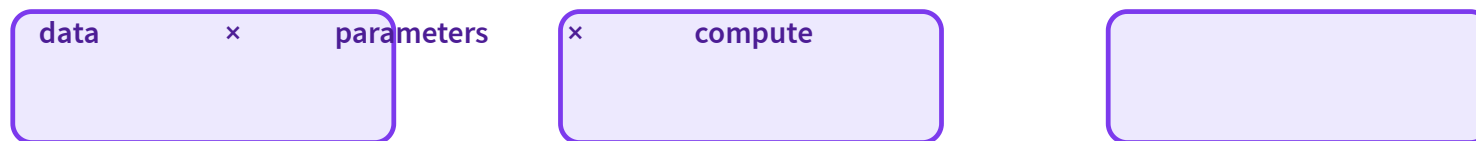
facts seen in data

skills

traces from
text and code

It learns statistical prediction, not guaranteed truth.

Pretraining at scale



Modern LMs improve when these are scaled carefully together.

Training is not inference

Training

many tokens in parallel
targets are known
weights update

Inference

one new token at a time
no target label
weights fixed

Decoding: from distribution to text

LIVE MODEL

A real next-token distribution; the knobs reshape it, then generate.

"To be, or not to"

"The robot picked up the cup because it was"

"Once upon a time, there was a"

temperature  1.0 top-k  5 top-p  1.00

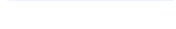
prompt: To be, or not to → ?

generate (real model)

· be  0.96

· ,  0.01

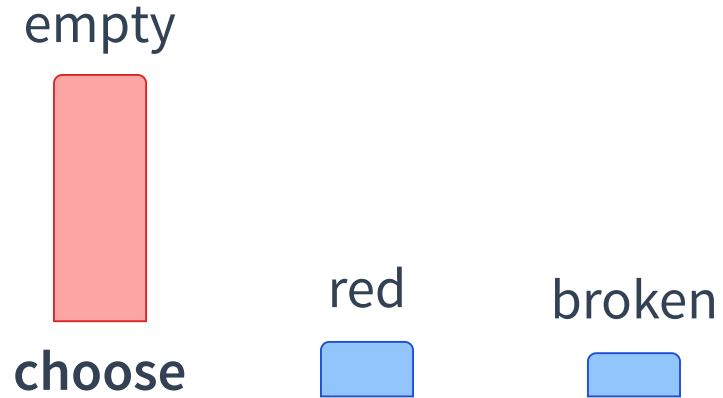
· have  0.01

· say  0.01

· .  0.01

Greedy decoding

Pick the highest-probability token each step.



Simple and stable, but often dull or brittle.

Sampling and temperature

Temperature reshapes
the distribution:

$$\text{softmax}(\text{logits}/T)$$

low T

sharper, safer, repetitive

high T

flatter, diverse, risky

Temperature, hands-on LIVE

Low temperature sharpens onto one token; high temperature flattens toward uniform.

temperature  1.0

dog  1.0

cat  3.0

bird  1.5

fish  0.5



Softmax turns scores into probabilities. Low temperature sharpens; high temperature flattens (the decoding knob in L20/L21).

Restrict randomness to plausible tokens

top-k

sample from the k
most likely tokens

top-p

sample from the
smallest set with
total probability $\geq p$

Decoding choices shape output even when weights are fixed.

What pretraining does not solve

truth

fluent text
can be wrong

instructions

base models
complete text

private data

not known
unless supplied

Later: prompting, fine-tuning, RAG, and tools.

Next: open the hood

Now we know:

- the architecture: causal transformer
- the objective: next-token prediction
- the inference step: sample the next token

L21: Dissecting a Small Language Model.

Recap

- ✓ Text becomes **tokens, IDs, embeddings**.
- ✓ A causal transformer predicts **next-token logits**.
- ✓ **Softmax + cross-entropy** train the model.
- ✓ Pretraining is **self-supervised** at scale.
- ✓ Decoding turns probabilities into generated text.